Agent Orchestration

Orchestrating Multiple Agent Sessions

Day 1 - Session 1, Topic 3

KavinSchool

What We'll Cover

1. Why multiple sessions create a coordination problem
2. Parallel sessions and scheduled Claude Code routines
3. Handoffs and integration gates
4. Demos: Worktrees and Handoff Gate
5. Lab 3.1: Parallel Agent Coordination

What Is an Agent Session?

An agent session is one running interaction with its own:

- Conversation and working context
- Assigned goal and instructions
- Available tools and permissions
- View of repository and workspace state

> Two sessions do not automatically share context, decisions, or uncommitted edits.

Why Use Multiple Sessions?

Separate sessions can work concurrently when tasks have stable boundaries.

- Reduce elapsed time for independent work
- Keep each session focused on a bounded task
- Apply different models, tools, or permissions where needed

Condition: The tasks must have low coupling and explicit integration points.

The Problem We Are Solving

Two agents implement email and Short Message Service (SMS) notifications at the same time.

- Both edit `router.py`, creating overwrite or merge risk.
- Each makes different assumptions about the shared schema.
- Focused tests pass, but the combined behavior fails.
- Results remain trapped in separate chat histories.

Need: Isolate work, transfer evidence, merge in dependency order, and test the combined state.

Task Card: Three Required Fields

PROVES DONE

Outcome

Named deliverable and acceptance check.

CONTROLS SCOPE

Boundary

Context, owned files, and non-goals.

CONNECTS WORK

Coordination

Dependencies and handoff evidence.

> One bounded unit, clear enough to delegate and verify.

The Delegation Chain

How high-level goals become executed tasks

KavinSchool

How Agents Invoke Each Other

Skill → Orchestrator → Workers

1 · SKILL

You run `/dev-factory` to start the workflow.

2 · ORCHESTRATOR

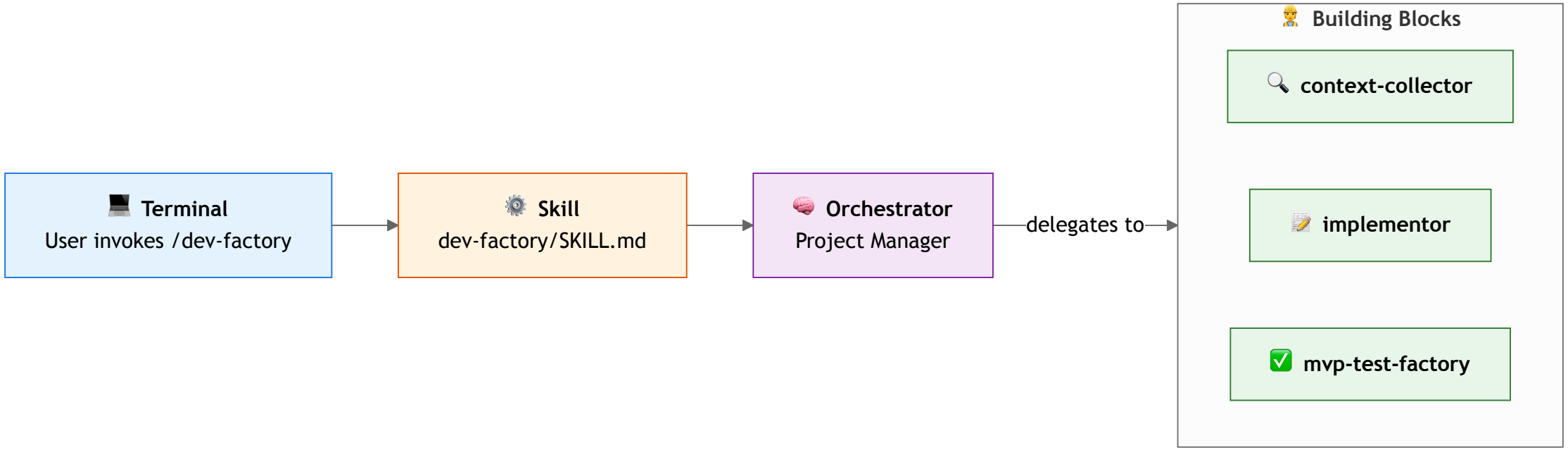
It follows the plan and decides which task runs next.

3 · WORKERS

Specialists complete bounded tasks in isolation.

> You start the chain. The orchestrator delegates the work.

Visualizing the Delegation Chain



> **Takeaway:** You invoke the Orchestrator. The Orchestrator invokes the workers.

> **Terminology:** In this course, "Workers", "Building Blocks", and "Subagents" all refer to the same thing: specialized agents spawned by an Orchestrator.

Interactive: Orchestrators and Building Blocks

A real toolkit example: follow the Dev Factory orchestrator from research through its TDD chain to the test-factory hand-off, then see how other orchestrators reuse the same reviewer, analyzer, and utility building blocks.

Open `orchestrator-building-block-map.html` from `claude-helper/docs/interactive/` to switch between the Dev Factory path view and the shared-blocks view.

Delegation Chain: Three Roles

ENTRY POINT

Skill

Reusable instructions that start a workflow.

COORDINATOR

Orchestrator

Coordinates the plan, tools, and workers.

WORKER

Subagent

Executes one bounded task in isolation.

Isolate the Workers

Separate files first, then coordinate evidence

KavinSchool

Choose the Coordination Surface

Approach	Coordinator	Best fit
Subagents	Main session	Focused side tasks
Agent view	Human operator	Independent monitored sessions
Agent teams	Lead agent	Experimental collaborative work
Dynamic workflows	Script	Large, repeatable multi-pass work

> Use the lightest mechanism that satisfies the coordination need.

What `/schedule` Schedules

A Routine saves a prompt, repositories, environment, connectors, and trigger.

- It runs in the cloud even when your laptop or terminal is off.
- Each trigger starts a Claude Code session that can use committed repository skills; scheduling alone does not define agent ownership or integration.
- Example: `/schedule daily PR review at 9am`.

`/loop` is session-local, needs an active session, and recurring loops expire after seven days.

Scheduled Work Needs Durable Access

1 · SIGN IN

`/schedule` requires a `claude.ai` subscription login.

2 · SCOPE ACCESS

Runs use your linked GitHub and connector identities.

3 · WATCH EXPIRY

GitHub access failures skip runs for up to 72 hours, then disable the routine.

4 · GATE IMPACT

Limit access and require human review before merges or deployments.

> Scheduled automation is only as durable as its identity and permissions.

Sources: Claude Code routines · Scheduled tasks

Concurrency Challenge

If two agents need to edit files in the same repository at the exact same time, what is the safest and most efficient way to prevent them from overwriting each other's uncommitted work?

A) Run them in the same directory but on different branches
B) Clone the entire repository twice into separate folders
C) Use Git Worktrees to share the repository history but isolate the files
D) Rely on the LLM to carefully merge lines

> **Chat Waterfall**
>
> **Type your answer in the chat, but do not send it yet.**
> Wait for the countdown: 3... 2... 1... BLAST

Concurrency Challenge: The Answer

C - Use Git Worktrees to share the repository history but isolate the files

Git worktrees provide complete isolation of uncommitted edits while remaining extremely fast and disk-efficient, making them perfect for parallel agent execution.

What is a Git Worktree?

A **Git Worktree** allows you to have multiple branches of the same repository checked out simultaneously in different folders on your machine.

Normally, you have one `.git` database and one set of working files. When you switch branches, the files change.
With a worktree, you keep **one** `.git` database, but create **multiple** independent working directories.

> Example: `git worktree add ../agent-ui-branch feature-ui`

Why Use Worktrees for Agents?

1. **Isolation:** Each agent gets its own directory. If Agent A creates a broken temporary file, Agent B never sees it.
2. **Efficiency:** Since they share the same underlying `.git` database, you don't waste time or disk space cloning the repository over and over.
3. **Simultaneous Branches:** Agent A can work on `feature-ui` while Agent B works on `feature-api`, concurrently, without context switching.

Worktrees Isolate Filesystem State

Each worktree has its own directory, index, branch, and uncommitted edits.

All worktrees share repository history and Git objects.

Isolation prevents accidental edit visibility across sessions.

It does not prevent incompatible assumptions or semantic conflicts.

What Does a Worktree Protect?

A worker has its own Git worktree. What does that isolation actually protect?

A) Dependency sequencing

B) Uncommitted edit visibility

C) Combined test success

D) Approval requirements

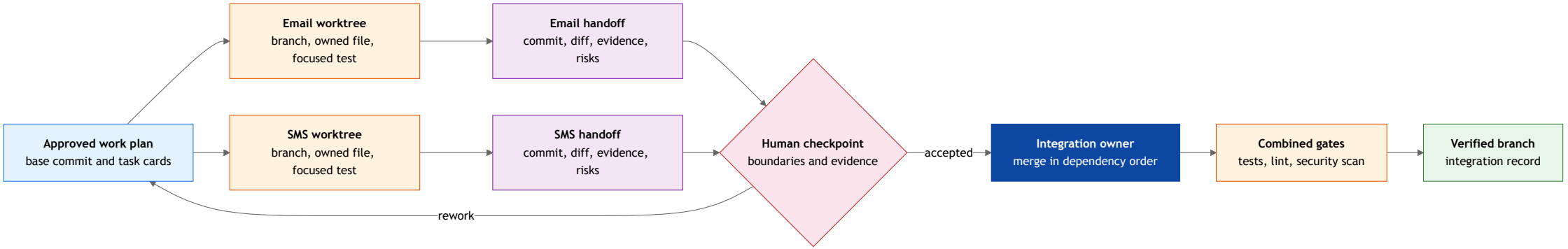
> **Chat Check:** Type your answer (A, B, C, or D) in the chat window now.

Worktree Protection: The Answer

B - Uncommitted edit visibility

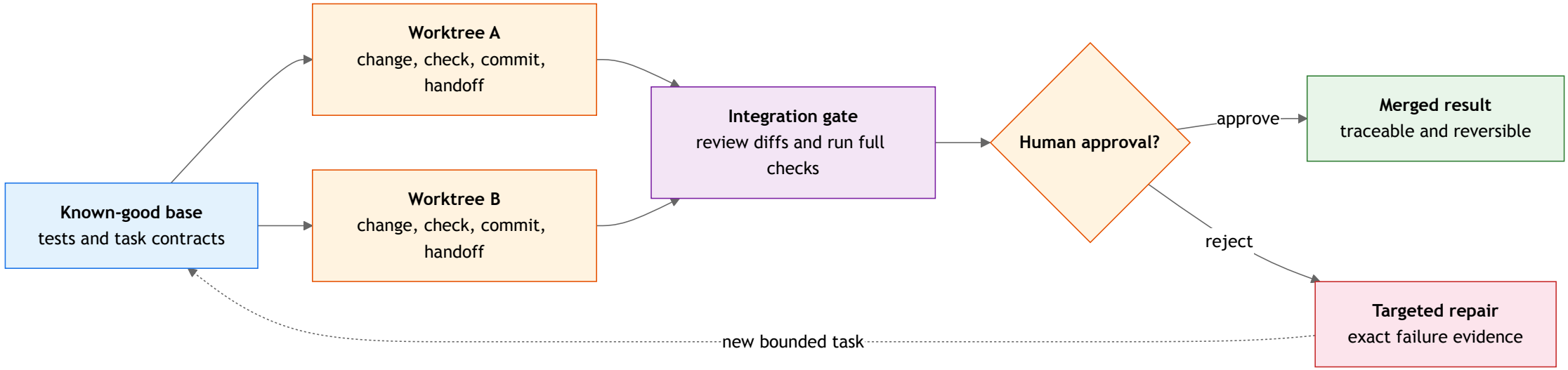
Each worktree isolates its directory, index, branch, and uncommitted edits. It does not prevent semantic conflicts or guarantee integrated behavior.

Parallel Worktree Orchestration



Generic Multi-Agent Worktree Flow

REFERENCE ONLY



Coordinated Specialist Agents (Financial Services)

REAL-WORLD SCENARIO

Financial services (Deloitte - Personal Wealth Advisor):

- Six specialist agents cover market data, portfolios, trades, advice, documents, and general assistance.
- The coordinated roles separate concerns while supporting recommendations and financial transactions.

What Defines an Agent's Workspace?

When we assign a task to an agent, we don't just give it a prompt; we give it a "workspace." What exactly does this workspace include?

A) Only the specific file it needs to edit

B) The LLM's context window and system prompt

C) The isolated environment (filesystem, compute, and credentials)

D) The main branch of the GitHub repository

> **Chat Waterfall**
>
> **Type your answer in the chat, but do not send it yet.**
>
> Wait for the countdown: 3... 2... 1... BLAST

What Defines an Agent's Workspace: The Answer

C - The isolated environment (filesystem, compute, and credentials)

A prompt tells the agent *what* to do. The workspace defines *where* it can do it and *what* it has access to.

Workspace Defines the Blast Radius

Filesystem + Compute + Credentials

FILESYSTEM

An isolated code checkout.

COMPUTE

Bounded processes, memory, and ports.

CREDENTIALS

Only the keys required for the task.

> An agent can only access what its workspace exposes.

The Need for Explicit Boundaries

When passing work between two human developers, we often rely on quick chats and shared intuition. When passing work between autonomous agents, what replaces this intuition to prevent cascading failures?

A) Providing longer, more detailed system prompts
B) Allowing the agents to chat with each other endlessly
C) Defining strict "Contracts" with verifiable inputs and outputs
D) Giving both agents full administrator access

> **Chat Waterfall**
>
> **Type your answer in the chat, but do not send it yet.**
> Wait for the countdown: 3... 2... 1... BLAST

The Need for Explicit Boundaries: The Answer

C - Defining strict "Contracts" with verifiable inputs and outputs

Agents lack human intuition. Without a strict contract, an agent might complete its task in a way that technically works but completely breaks the downstream integration.

Contracts Make “Done” Verifiable

An agentic contract defines one phase’s inputs, constraints, and expected outputs.

CLARIFY

Workers know the boundary and success condition.

VERIFY

Machine-readable outputs let the orchestrator check success.

CONTAIN

Invalid output stops before it reaches downstream work.

Next: workspace assignment and durable handoff contracts.

Workspace Assignment Contract

1. Base commit, branch, and worktree path
2. Task-card identifier and dependency state
3. Allowed files and prohibited shared files
4. Focused verification command
5. Integration owner and approval checkpoint

All parallel workers start from the same approved base.

Demo: Git Worktree Isolation

REFERENCE ONLY

From the companion repo, run `python3 day1/concepts/demos-conceptual/demo-git-worktree-isolation.py`.

- Git evidence: two real worktrees and branches are created.
- Isolation: worker files remain absent from `main` before merge.
- Integration: explicit merges create the combined history.

> Uses temporary directories, Git, and the Python standard library.

Demo: Two Coordinated Sessions End to End

From the companion repo, run `python3 day1/demos/demo-two-session-orchestration.py`.

- Isolation: two real worktrees; `main` stays at the base commit.
- Handoffs: each records its real commit and real focused-test result.
- Ordered merge: one human approval point before the second merge.
- Conflict: Git reports `UU router.py`; the approved order wins and stays green.

> Requires Git; runs in a temporary clone with no API key.
> Notebook: `day1/demos-notebook/demo-two-session-orchestration.ipynb`.

Demo: Orchestrating an MVP Test Factory

From the companion repo, open `day1/demos/mvp-test-factory/README.md`.

- **Goal:** Validate an MVP (`password-strength-checker`) by orchestrating API, UI, and Load testing agents simultaneously.
- **Observe:** How agents parse the test boundaries and operate concurrently without conflicting.
- **Evidence:** Review the separate handoffs and combined test gates before merging the generated code.

> Requires `claude-helper` installed and a local clone of `password-strength-checker`.

Admit Evidence, Not Claims

Handoffs and checks control integration

KavinSchool

Durable Handoff Contract

Evidence	Why integration needs it
Base and resulting commits	Reconstruct the exact patch lineage.
Changed files	Confirm ownership boundaries.
Decisions and assumptions	Expose effects on other tasks.
Focused command and result	Verify the bounded outcome.
Risks and next checkpoint	Decide whether work can advance.

Why Record Both Commits?

Why should a durable handoff include the base and resulting commits?

A) Calculate token cost

B) Confirm effort level

C) Prove patch lineage

D) Avoid all conflicts

> **Chat Check:** Type your answer (A, B, C, or D) in the chat window now.

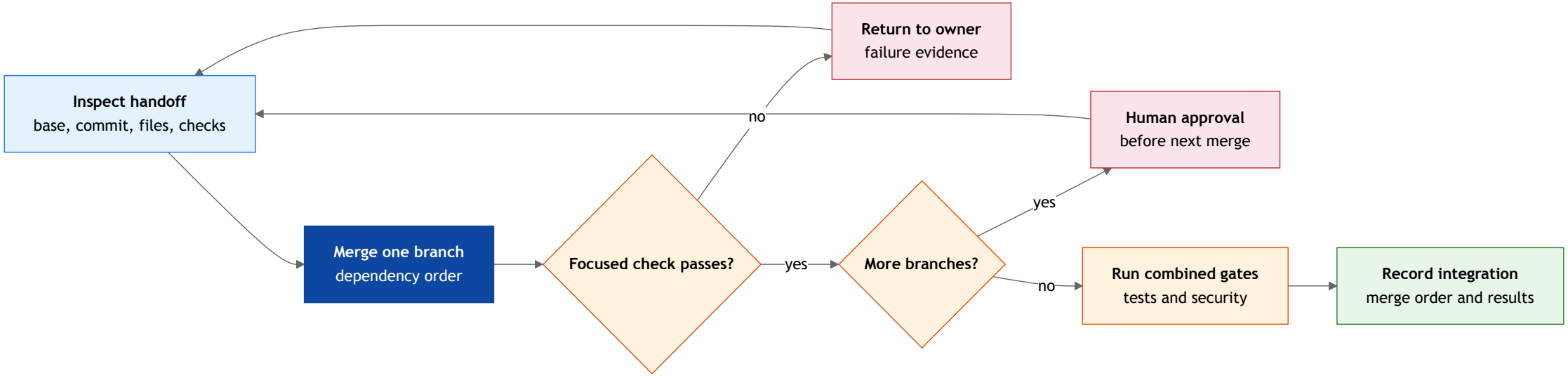
Commit Evidence: The Answer

C - Prove patch lineage

The two commits prove the approved starting point and identify the exact resulting patch.

They do not prevent merge conflicts.

Integrate in Dependency Order



A Clean Merge Is Not Compatibility

Git detects overlapping text edits.

Tests detect some behavioral incompatibilities.

Security scanning detects a different class of integration risk.

Human review checks architecture, authorization, and business impact.

Use focused checks after each merge and full gates on the combined state.

Why Run Full Gates?

Each branch passed focused tests and merged cleanly. Why test the combined state?

A) Git requires it

B) It uses live data

C) Catch combined-state risks

D) Agents cannot test

> **Chat Check:** Type your answer (A, B, C, or D) in the chat window now.

Full Gates: The Answer

C - Catch combined-state risks

Focused tests prove bounded outcomes. Full gates catch semantic incompatibilities and security failures that appear only after integration.

Demo: Handoff Integration Gate

REFERENCE ONLY

From the companion repo, run `python3 day1/concepts/demos-conceptual/demo-handoff-integration-gate.py`.

- Base gate: both workers must start from the approved commit.
- Ownership gate: a router edit blocks a channel handoff.
- Risk gate: passing tests do not erase an unresolved concern.

> Runs locally without an API key or network access.

Demo (Optional): Handoff Summary Without a Key

REFERENCE ONLY

From the companion repo, run `python3 day1/demos/demo-handoff-summary.py`.

- Structural fields measured from Git and tests; only the prose is generated.
- A small local model (`flan-t5-base`) summarizes the real diff.
- Model output is rejected if it echoes the diff, then falls back cleanly.
- `summary_source` records which path produced the field.

> Optional and skippable. Completes without the model or `transformers`.
> Notebook: `day1/demos-notebook/demo-handoff-summary.ipynb`.

Enterprise Translation

Local course control	Enterprise control
Git worktree	Ephemeral cloud workspace
Short-lived branch	Trunk-based pull request
Local acceptance command	Continuous integration required check
Security unit test	Static and dynamic security scanning
Integration owner	Protected branch or merge queue

What Industry Experiments Show

Anthropic's compiler experiment used 16 agents and nearly 2,000 sessions.

Progress depended on tests, task locks, isolated environments, and clean feedback.

Parallelism stopped helping when every agent encountered the same kernel failure.

The harness had to expose independent subsets before workers could specialize.

> This is a case study, not a general productivity guarantee.

Automating the Handoff

TRICK OF THE TRADE

In practice, humans don't manually merge branches from JSON files. We use output parsing.

Extract the downstream target branch from Agent 1's handoff using `jq`:

```
TARGET_COMMIT=$(jq -r '.implementation_commit' handoffs/email.json)
git merge $TARGET_COMMIT
```

> **Pro Tip:** Structured JSON outputs are only useful if you use CLI tools to parse and pipe them into your Git operations automatically.

CI/CD as the Orchestrator

TRICK OF THE TRADE

The real orchestrator is your CI/CD pipeline (GitHub Actions, GitLab CI).

1. Agent opens a Pull Request.
2. CI runs the integration gate (`make test`).
3. If failed, a webhook comments the error log back onto the PR.
4. The Agent automatically picks up the comment and pushes a fix.

> **Pro Tip:** Stop treating the terminal as the only orchestration layer.

Lab 3.1: Parallel Agent Coordination

Open `day1/lab3_1_parallel-agent-coordination/` in the companion repo.

Goal: Complete two channel workers in isolated Claude Code sessions.

1. Create worktrees and enforce non-overlapping ownership.
2. Review commits, diffs, checks, and handoffs.
3. Approve, merge in order, run combined gates, and resolve the fixture.

> Zero external dependencies; each lab starts from a fresh repository.

Key Takeaways

1. Worktrees isolate edits but do not solve logical coordination.
2. Every workspace needs a common base, exclusive scope, and focused check.
3. Durable handoffs carry commits, diffs, evidence, risks, and checkpoints.
4. Integration follows dependencies and verifies the combined state.
5. Human approval remains essential at security and high-risk boundaries.

Course Wrap-Up

Plan, route, coordinate, and verify

KavinSchool

Day 1 Outcomes

You can now:

1. Decompose a feature into bounded, verifiable task cards.
2. Select model and effort from complexity, risk, and cost evidence.
3. Coordinate isolated sessions with explicit ownership and handoffs.
4. Integrate in dependency order with automated and human gates.

Demo: Interactive Multi-Agent Coordination

Open `day1/demos/interactive-multi-agent-coordination.html` from the companion repository.

- Compare Subagents, Agent view, agent teams, and dynamic workflows.
- Identify who coordinates each surface and when it fits.
- Inspect the workspace, handoff, and integration controls that every surface still requires.

Select a coordination surface, then trace its relationship to the shared controls.

> Built with Archify from `interactive-multi-agent-coordination.architecture.json`. Self-contained; no server, install, API key, or network required.

Put It to Work

Complete this privately:

> When [specific workplace situation] occurs, I will [orchestration practice], using [artifact or evidence], and I will know it worked when [observable result].

Exchange plans with one partner. Ask:

1. Which boundary, dependency, or approval condition is explicit?
2. Which check proves the result?

Revise one line. Keep a private note instead if you prefer.

Questions?

Keep the plan and evidence durable after the sessions close.